

MERN STACK PRACTICAL EXAMINATION

Course Management System

Duration: 3 Hours

Problem Statement

Develop a **Course Management System** using the MERN Stack. The application should allow an administrator to manage courses and students and enroll students into courses.

Technology Requirements

- Frontend: React.js
- Backend: Node.js + Express.js
- Database: MongoDB
- API: REST API
- HTTP Client: Axios

1. Course Management

Course Fields

Course Name
Description
Duration
Course Fee
Status

Requirements

- Add a course
- Display all courses
- Edit a course
- Delete a course
- Search courses by name
- Filter courses by status

Status: Active / Inactive

2. Student Management

Student Fields

Student Name
Email
Phone
Qualification
Course
Status

Requirements

- Add student
- Display all students
- Edit student
- Delete student
- Search students by name
- Filter students by course

Status: Active / Inactive

3. Backend Requirements

Course APIs

```
POST /api/courses
GET /api/courses
GET /api/courses/:id
PUT /api/courses/:id
DELETE /api/courses/:id
```

Student APIs

```
POST /api/students
GET /api/students
GET /api/students/:id
PUT /api/students/:id
DELETE /api/students/:id
```

4. MongoDB Requirements

Create two collections: **courses** and **students**. Use Mongoose for database operations.

Course Schema

name, description, duration, fee, status

Student Schema

name, email, phone, qualification, course, status

The student's **course** should reference the appropriate course.

5. React Frontend

Create a React application with the following pages/components:

- Dashboard
- Courses
- Students

Dashboard

Display total courses, active courses, total students, and active students. Use API data for the counts.

6. Form Validation

Course: Course name is required; duration is required; fee must be a valid number; status is required.

Student: Student name is required; email must be valid; phone is required; course is required.

Display suitable validation/error messages.

7. Search & Filter Functionality

Courses: Search by course name and filter by status.

Students: Search by student name and filter by course.

Search/filter results should update the displayed data correctly.

8. UI Requirements

The application should have:

- Navigation bar/sidebar
- Tables for displaying data
- Add/Edit forms
- Delete confirmation
- Search box
- Filter dropdown
- Loading state
- Error handling
- Success/error messages

The UI does not need to be highly designed. **Functionality is more important.**

9. API Integration

React should communicate with the Express backend using Axios. Do not use static/hardcoded data for the main CRUD operations.

React → Axios → Express API → Mongoose → MongoDB

10. Bonus Task

If all mandatory requirements are completed, implement **one** of the following:

- Pagination
- Course-wise student count
- Sorting
- Login page
- Dashboard chart

Bonus: 5 marks

Exam Rules

1. Create a separate GitHub repository for the project.
2. Use meaningful component and variable names.
3. Follow proper folder structure.
4. Do not use copied complete projects.
5. Students should be able to explain their code.
6. Backend and frontend must communicate through REST APIs.
7. Database operations must be performed using MongoDB.
8. All major CRUD operations should work before submission.

Suggested Folder Structure

```
course-management/  
  ■■■ client/  
    ■ ■■■ src/  
      ■ ■■■ components/  
      ■ ■■■ pages/  
      ■ ■■■ services/  
      ■ ■■■ App.jsx  
    ■■■ server/  
      ■■■ models/  
      ■■■ routes/  
      ■■■ controllers/  
      ■■■ middleware/  
      ■■■ server.js
```

Expected Result

At the end of the examination, the student should have a working application where an administrator can manage courses and students, search/filter records, and view course/student counts on the dashboard.

Focus: Demonstrate practical understanding of React.js, Node.js, Express.js, MongoDB, Mongoose, REST APIs, Axios, CRUD operations, validation, search, and filtering.